

Лабораторная работа №8

Перенаправление ввода-вывода, поиск файлов и управление процессами

Давид Майкл Фрэнсис

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Перенаправление ввода-вывода, поиск файлов и управление процессами».

Рассмотрены потоки ввода-вывода и операторы их перенаправления, конвейеры, поиск и фильтрация файлов с помощью `find` и `grep`, управление процессами и фоновыми задачами, а также проверка использования дискового пространства.

Сделаны выводы о достижении поставленной цели.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
1.4	Объект и предмет исследования	7
1.5	Методы исследования	7
1.6	Схема выполнения работы	7
2	Теоретическая часть	9
2.1	Перенаправление ввода-вывода	9
2.2	Конвейер	9
2.3	Поиск файлов	10
2.4	Управление процессами и задачами	10
3	Материалы и методы	11
4	Выполнение лабораторной работы	12
4.1	Перенаправление вывода и поиск по содержимому файлов	12
4.2	Управление фоновыми процессами	14
4.3	Проверка использования диска	16
5	Результаты	18
6	Обсуждение результатов	19
7	Заключение	20
	Ответы на контрольные вопросы	21
	Список литературы	24

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	8
4.1	Фильтрация файлов с расширением .conf	13
4.2	Поиск файлов, начинающихся с символа с	14
4.3	Постраничный просмотр файлов, начинающихся с h	14
4.4	Запуск gedit в фоновом режиме	15
4.5	Определение PID процесса gedit	16
4.6	Проверка использования диска (df, du)	17

Список таблиц

1 Введение

1.1 Актуальность

Перенаправление ввода-вывода, конвейеры и инструменты поиска файлов составляют основу эффективной работы в командной строке Linux, позволяя объединять простые утилиты в гибкие цепочки обработки данных. Управление процессами и фоновыми задачами, а также контроль использования дискового пространства — повседневные практические задачи администрирования системы, без которых невозможна полноценная работа в многозадачной среде.

1.2 Цель работы

Ознакомление с инструментами поиска файлов и фильтрации текстовых данных. Приобретение практических навыков по управлению процессами (и заданиями), по проверке использования диска и обслуживанию файловых систем.

1.3 Задачи

- изучить потоки ввода-вывода и операторы их перенаправления;
- освоить использование конвейеров для объединения команд;
- освоить поиск файлов с помощью `find` и фильтрацию текста с помощью `grep`;

- освоить управление процессами и фоновыми задачами (`ps`, `kill`, `jobs`, `pgrep`, `killall`);
- освоить проверку использования диска командами `df` и `du`.

1.4 Объект и предмет исследования

Объектом исследования являются средства перенаправления ввода-вывода, поиска файлов и управления процессами командной строки Linux. Предметом исследования выступает процесс поиска и фильтрации файлов, управления процессами и фоновыми задачами, а также анализа использования дискового пространства.

1.5 Методы исследования

Работа выполнялась практическим методом — путём непосредственного взаимодействия с интерпретатором команд операционной системы Linux.

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий при выполнении работы.

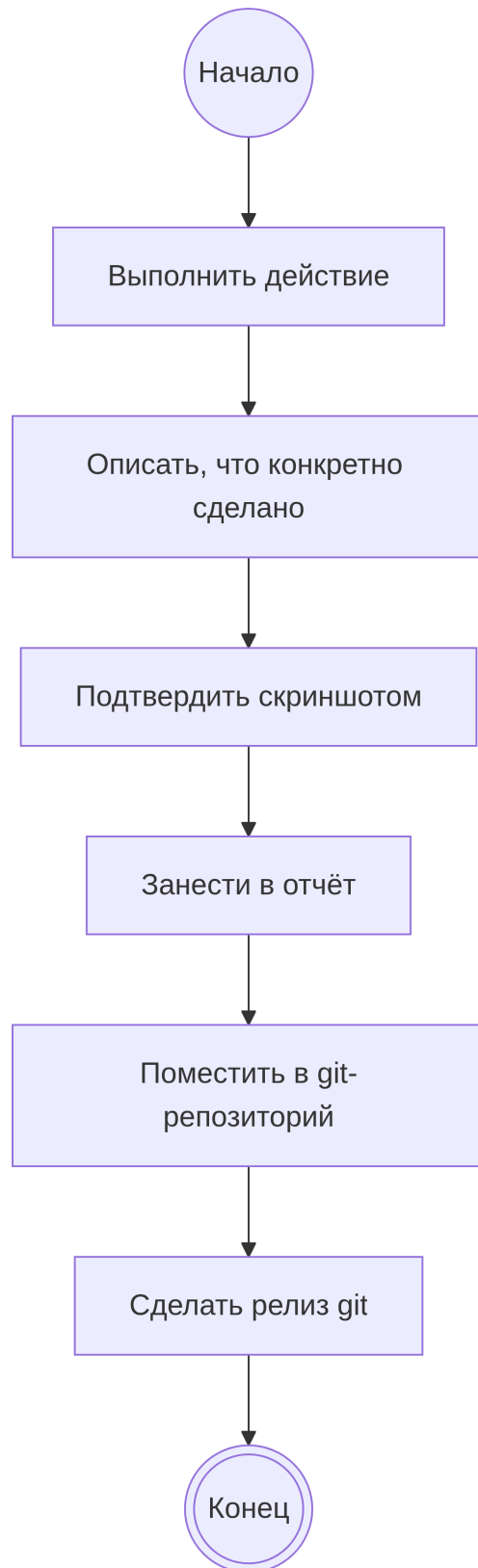


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

2.1 Перенаправление ввода-вывода

В системе по умолчанию открыто три специальных потока: **stdin** — стандартный поток ввода (по умолчанию клавиатура, дескриптор 0), **stdout** — стандартный поток вывода (по умолчанию консоль, дескриптор 1) и **stderr** — стандартный поток вывода ошибок (по умолчанию консоль, дескриптор 2) [1]. Основные операторы перенаправления:

Оператор	Описание
>	Перенаправление вывода в файл (перезапись)
>>	Перенаправление вывода в файл (добавление)
<	Перенаправление ввода из файла
2>	Перенаправление stderr в файл
&>	Перенаправление stdout и stderr в файл

2.2 Конвейер

Конвейер (pipe) служит для объединения команд в цепочки, в которых результат работы предыдущей команды передаётся на вход последующей: `cat file1 | grep pattern2`. Это позволяет комбинировать простые, узкоспециализированные утилиты для решения более сложных задач обработки данных.

2.3 Поиск файлов

Команда `find` `Пат` [`-Пат`] используется для поиска файлов по заданным критериям (имени, типу, времени изменения и др.) с рекурсивным обходом каталогов. Команда `grep` `Пат` `Пат_Пат` используется для фильтрации текста — поиска строк, соответствующих заданному шаблону, в содержимом файлов.

2.4 Управление процессами и задачами

Команда `ps aux` выводит информацию о запущенных в системе процессах, `kill PID` завершает процесс по его идентификатору, а `jobs` выводит список фоновых задач текущей сессии терминала. Для анализа использования дискового пространства применяются команды `df` (свободное место на файловых системах) и `du` (размер файлов и каталогов).

3 Материалы и методы

Для выполнения работы использовались операционная система семейства Linux и команды `find`, `grep`, `ps`, `pgrep`, `kill`, `killall`, `jobs`, `df`, `du`, а также операторы перенаправления ввода-вывода (`>`, `>>`, `|`).

4 Выполнение лабораторной работы

4.1 Перенаправление вывода и поиск по содержимому файлов

Вход в систему был подтверждён командой `whoami`. Далее содержимое каталога `/etc` было записано в файл `file.txt` с перезаписью, а содержимое домашнего каталога — дописано в тот же файл:

```
whoami  
ls /etc > file.txt  
ls ~ >> file.txt
```

Из полученного файла с помощью `grep` были отфильтрованы строки, содержащие файлы с расширением `.conf`, и результат сохранён в `conf.txt` (рис. 4.1):

```
grep "\.conf" file.txt  
grep "\.conf" file.txt > conf.txt
```

```
(base) mike@DESKTOP-I4EKT0:~/study_2025-2026_os-intro/labs/lab07$ gre
p "\.conf" file.txt
adduser.conf
app.conf
ca-certificates.conf
debconf.conf
deluser.conf
dhcpcd.conf
e2scrub.conf
fuse.conf
gai.conf
gssapi_mech.conf
host.conf
insserv.conf.d
ld.so.conf
ld.so.conf.d
libaudit.conf
locale.conf
logrotate.conf
manpath.config
mke2fs.conf
nsswitch.conf
pam.conf
resolv.conf
rsyslog.conf
rygel.conf
sensors3.conf
sudo.conf
sudo_logsrvd.conf
sysctl.conf
ucf.conf
updatedb.conf
usb_modeswitch.conf
vconsole.conf
wsl.conf
xattr.conf
```

Рисунок 4.1: Фильтрация файлов с расширением .conf

Файлы домашнего каталога, начинающиеся с символа `c`, были найдены тремя разными способами — с помощью `find`, подстановки имён файлов в `ls` и в `echo` (рис. 4.2):

```
# 1 — find
find ~ -name "c*"

# 2 — ls
ls ~/c*

# 3 — echo
echo ~/c*
```

```
cascmd_en/cascmd_en494.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en352.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en878.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en310.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en812.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en092.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en558.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en720.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en342.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en331.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en551.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en541.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en241.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en264.html
/home/mike/miniforge3/pkgs/giac-1.9.0.21-h673759e_1/share/giac/doc/en/
cascmd_en/cascmd_en613.html
```

Рисунок 4.2: Поиск файлов, начинающихся с символа c

Файлы каталога /etc, начинающиеся с символа h, были выведены постранично с помощью конвейера из ls, grep и less (рис. 4.3):

```
ls /etc | grep "^h" | less
```

```
host.conf
hostname
hosts
hosts.allow
hosts.deny
(END)
```

Рисунок 4.3: Постраничный просмотр файлов, начинающихся с h

4.2 Управление фоновыми процессами

Был запущен фоновый процесс поиска файлов log* по всей файловой системе с записью результата в ~/logfile, после чего созданный файл был удалён:

```
find / -name "log*" > ~/logfile &  
rm ~/logfile
```

Далее в фоновом режиме был запущен текстовый редактор `gedit` (рис. 4.4):

```
gedit &
```

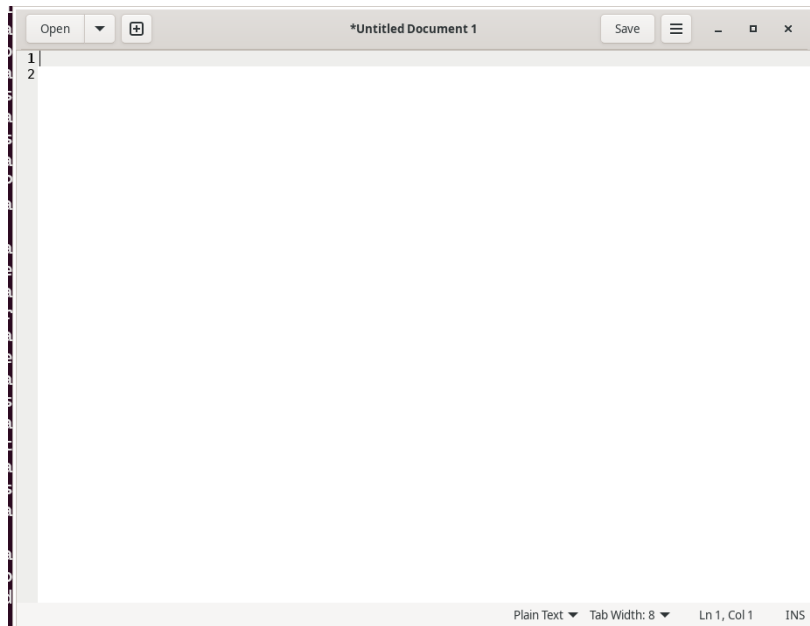


Рисунок 4.4: Запуск `gedit` в фоновом режиме

Идентификатор процесса `gedit` был определён двумя способами — через конвейер `ps aux | grep` и напрямую через `pgrep` (рис. 4.5):

```
# 🔍 🔍 ps 🔍 grep  
ps aux | grep gedit  
# 🔍 🔍 pgrep  
pgrep gedit
```

```
(base) mike@DESKTOP-I4EKT0:~/study_2025-2026_os-intro/labs/lab07$ ps
aux | grep gedit
mike      957   1.1  1.3 808760 105552 pts/2    Sl   19:57   0:00  ged
it
mike      1053   0.0  0.0  4092   1908 pts/2    S+   19:58   0:00  gre
p --color=auto gedit
```

Рисунок 4.5: Определение PID процесса gedit

Процесс `gedit` был завершён с помощью `kill` по идентификатору процесса либо с помощью `killall` по имени процесса, после предварительного изучения справки:

```
man kill
kill PID
killall gedit
```

4.3 Проверка использования диска

С помощью `man` были изучены команды `df` и `du`, после чего было проверено использование дискового пространства и размер домашнего каталога (рис. 4.6):

```
man df
man du
df -vi
du -a ~/
```

```
(base) mike@DESKTOP-I4EKT0:~/study_2025-2026_os-intro/labs/lab07$ df
Filesystem            Inodes  IUsed   IFree IUse% Mounted on
none                   972184    4    972180    1% /usr/lib/modules/5.15.1
53.1-microsoft-standard-WSL2
none                   972184    2    972182    1% /mnt/wsl
drivers                 999 -999001 1000000    - /usr/lib/wsl/drivers
/dev/sdc               67108864 559911 66548953    1% /
none                   972184    40    972144    1% /mnt/wslg
none                   972184    6    972178    1% /usr/lib/wsl/lib
rootfs                 971330    12    971318    1% /init
none                   972184   599    971585    1% /run
none                   972184    2    972182    1% /run/lock
none                   972184    1    972183    1% /run/shm
tmpfs                  1024     19     1005    2% /sys/fs/cgroup
none                   972184   150    972034    1% /mnt/wslg/versions.txt
none                   972184   150    972034    1% /mnt/wslg/doc
C:\                     999 -999001 1000000    - /mnt/c
snapfuse               10869   10869    0   100% /snap/hugo/26009
snapfuse               10549   10549    0   100% /snap/core24/1587
snapfuse               10545   10545    0   100% /snap/core24/1643
snapfuse               10869   10869    0   100% /snap/hugo/26124
snapfuse                610     610    0   100% /snap/snapd/26865
tmpfs                 194436    82   194354    1% /run/user/1000
```

Рисунок 4.6: Проверка использования диска (df, du)

Завершающим шагом были выведены все вложенные каталоги домашнего каталога:

```
find ~ -type d
```

5 Результаты

В результате выполнения работы:

- отработаны операторы перенаправления вывода (`>`, `>>`) и использование конвейеров;
- освоена фильтрация текста с помощью `grep` и поиск файлов с помощью `find`, а также альтернативные способы поиска через подстановку имён файлов;
- запущены и исследованы фоновые процессы, определены их идентификаторы (PID) и выполнено их завершение;
- проверено использование дискового пространства и размер домашнего каталога с помощью `df` и `du`.

6 Обсуждение результатов

Сравнение трёх способов поиска файлов, начинающихся с `c` (`find`, подстановка имён файлов в `ls`, подстановка в `echo`), наглядно показывает различие между специализированным инструментом рекурсивного поиска (`find`) и механизмом подстановки имён файлов самой оболочки (`glob`), который ограничен текущим каталогом и синтаксисом шаблонов оболочки. Запуск `gedit` и поиска лог-файлов в фоновом режиме с последующим определением их PID через `ps/pgrep` и завершением через `kill/killall` демонстрирует полный практический цикл управления процессом — от запуска до завершения.

7 Заключение

В ходе выполнения лабораторной работы были изучены инструменты поиска файлов и фильтрации текстовых данных в ОС Linux. Были получены практические навыки работы с командами `find`, `grep`, `ps`, `kill`, `df` и `du`. Изучены механизмы перенаправления ввода-вывода и работы с конвейерами. Получены навыки управления процессами и фоновыми задачами.

Поставленная цель работы достигнута.

Ответы на контрольные вопросы

1. Потоки ввода-вывода

- **stdin** — стандартный поток ввода (по умолчанию клавиатура), файловый дескриптор 0;
- **stdout** — стандартный поток вывода (по умолчанию консоль), файловый дескриптор 1;
- **stderr** — стандартный поток вывода ошибок (по умолчанию консоль), файловый дескриптор 2.

2. Разница между > и >>

- > — перенаправляет вывод в файл, **перезаписывая** его, если он уже существует;
- >> — перенаправляет вывод в файл в режиме **добавления**, не перезаписывая существующее содержимое.

3. Что такое конвейер

Конвейер (pipe) — механизм объединения команд в цепочки, где вывод одной команды передаётся на ввод следующей с помощью символа `|`. Например, `ls -la | sort` передаёт вывод команды `ls` команде `sort`.

4. Что такое процесс и чем он отличается от программы

Программа — это статический набор инструкций, хранящихся на диске. Процесс — это запущенный экземпляр программы в памяти с собственными выделенными

ресурсами, адресным пространством и идентификатором. Из одной программы одновременно может быть запущено несколько процессов.

5. Что такое PID и GID

- **PID (Process ID)** — уникальный числовой идентификатор, присваиваемый каждому запущенному процессу в системе;
- **GID (Group ID)** — числовой идентификатор группы, к которой принадлежит процесс или пользователь.

6. Что такое задачи и какая команда управляет ими

Задачи — это программы или команды, выполняющиеся в фоновом режиме терминала. Ими управляют с помощью команды `jobs`, которая выводит список всех текущих фоновых задач. Задачу можно завершить командой `kill %XXXXXX_XXXXXX`.

7. Утилиты `top` и `htop`

- **top** — встроенная утилита Linux, отображающая в реальном времени информацию о запущенных процессах, использовании CPU, памяти и нагрузке на систему;
- **htop** — улучшенная интерактивная версия `top` с более удобным интерфейсом, цветным отображением и поддержкой мыши, позволяющая легче управлять процессами.

8. Команда поиска файлов и примеры

Команда `find` используется для поиска файлов:

```
# Найти файлы, начинающиеся с f, в текущем каталоге
find ~ -name "f*"

# Найти файлы в /etc, начинающиеся с p
find /etc -name "p*"

# Найти файлы, начинающиеся с ~, в текущем каталоге
find ~ -name "*~" -exec rm "{}" \;
```

```
# root@kali:~# find ~ -type d
```

9. Можно ли найти файл по содержимому

Да, с помощью команды `grep`:

```
# root@kali:~# grep -r "hello" ~
# root@kali:~# grep -rl "hello" /root/
```

10. Как определить объём свободной памяти на жёстком диске

```
df -h
```

Флаг `-h` отображает размеры в удобочитаемом формате (КБ, МБ, ГБ).

11. Как определить объём домашнего каталога

```
du -sh ~
```

Флаг `-s` показывает общий размер, а `-h` отображает его в удобочитаемом формате.

12. Как удалить зависший процесс

```
# root@kali:~# ps aux | grep _
# root@kali:~# kill PID
# root@kali:~# kill -9 PID
# root@kali:~# killall _
```

Список литературы

1. *Shotts W. E.* The Linux Command Line: A Complete Introduction. — 2nd ed. — No Starch Press, 2019. — ISBN 978-1593279523.